

S&DS 365 / 665
Intermediate Machine Learning

Approximate Inference: Simulation and Variational Methods

October 1

Yale

Reminders

- Quiz 3 available today at 2:30pm
 - ▶ Available for 48 hours
 - ▶ 20 minutes once started
 - ▶ DD, CNNs, GPs
- Assignment 2 due week from tomorrow
- Midterm review sessions
 - ▶ Thursday Oct 9 (Keyi, 4pm KT207)
 - ▶ Friday Oct 10 (Awni, 5pm KT1101)
 - ▶ Saturday Oct 11 (TBD)

Another reminder

For all course business, please email sds365@yale.edu

For Today

- Finish up Gaussian processes
- Define the Dirichlet process
- Where are we headed? Road map for next couple classes
- Approximate inference: What's it all about?
- Approximate inference with Gibbs sampling
- Variational methods:
 - ▶ Mean field approximation
 - ▶ Two examples

Gaussian processes

The key to computation in the Gaussian process is *conjugacy*: If the noise model is Gaussian, the posterior is another Gaussian process.

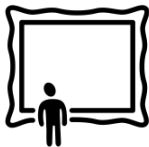
- Everything boils down to multivariate Gaussian calculations

For classification, the likelihood is not Gaussian. We don't have conjugacy and the computations become more complex.

- We need to carry out some sort of *approximation*

Where have we gone, where are we headed?

Let's pause to discuss the "big picture"





Different approaches

	Frequentist	Bayesian
Probability	limiting frequency	degree of subjective belief
Parameter θ :	fixed constant	random variable
Statements are about:	procedures	parameters
Frequency guarantees?	yes	no



Correspondence

Statistical problem	Frequentist approach	Bayesian approach
regression	kernel smoother	Gaussian process
CDF estimation	empirical cdf	Dirichlet process
density estimation	kernel density estimator	Dirichlet process mixture
regression	wide neural network	Gaussian process



Bayesian computation

- Computing Bayesian posteriors can be difficult
- Two approaches: Simulation and variational
- Simulation is the “right” way to do it — unbiased
- Variational methods are an alternative, preferred in ML



Inverting generative models

Template for generative model:

- 1 Choose Z
- 2 Given z , generate (sample) X

We often want to invert this:

- 1 Given x
- 2 What is Z that generated it?



Inverting models

Bayesian setup:

- 1 Choose θ
- 2 Given θ , generate (sample) X

Posterior inference:

- 1 Given x
- 2 What is θ that generated it?



Approximate inference

If we have a random vector $Z \sim p(Z | x)$, we might want to compute the following:

- marginal probabilities $\mathbb{P}(Z_i = z | x)$



Approximate inference

If we have a random vector $Z \sim p(Z | x)$, we might want to compute the following:

- marginal probabilities $\mathbb{P}(Z_i = z | x)$
- marginal means $\mathbb{E}(Z_i = z | x)$



Approximate inference

If we have a random vector $Z \sim p(Z \mid x)$, we might want to compute the following:

- marginal probabilities $\mathbb{P}(Z_i = z \mid x)$
- marginal means $\mathbb{E}(Z_i = z \mid x)$
- most probable assignments $z^* = \arg \max_z \mathbb{P}(\{Z_i = z_i\} \mid x)$



Approximate inference

If we have a random vector $Z \sim p(Z | x)$, we might want to compute the following:

- marginal probabilities $\mathbb{P}(Z_i = z | x)$
- marginal means $\mathbb{E}(Z_i = z | x)$
- most probable assignments $z^* = \arg \max_z \mathbb{P}(\{Z_i = z_i\} | x)$
- maximum marginals $z_i^* = \arg \max_{z_i} \mathbb{P}(Z_i = z_i | x)$



Approximate inference

If we have a random vector $Z \sim p(Z | x)$, we might want to compute the following:

- marginal probabilities $\mathbb{P}(Z_i = z | x)$
- marginal means $\mathbb{E}(Z_i = z | x)$
- most probable assignments $z^* = \arg \max_z \mathbb{P}(\{Z_i = z_i\} | x)$
- maximum marginals $z_i^* = \arg \max_{z_i} \mathbb{P}(Z_i = z_i | x)$
- joint probability $\mathbb{P}(Z | x)$



Approximate inference

If we have a random vector $Z \sim p(Z \mid x)$, we might want to compute the following:

- marginal probabilities $\mathbb{P}(Z_i = z \mid x)$
- marginal means $\mathbb{E}(Z_i = z \mid x)$
- most probable assignments $z^* = \arg \max_z \mathbb{P}(\{Z_i = z_i\} \mid x)$
- maximum marginals $z_i^* = \arg \max_{z_i} \mathbb{P}(Z_i = z_i \mid x)$
- joint probability $\mathbb{P}(Z \mid x)$
- joint mean $\mathbb{E}(Z \mid x)$



Approximate inference

If we have a random vector $Z \sim p(Z \mid x)$, we might want to compute the following:

- marginal probabilities $\mathbb{P}(Z_i = z \mid x)$
- marginal means $\mathbb{E}(Z_i = z \mid x)$
- most probable assignments $z^* = \arg \max_z \mathbb{P}(\{Z_i = z_i\} \mid x)$
- maximum marginals $z_i^* = \arg \max_{z_i} \mathbb{P}(Z_i = z_i \mid x)$
- joint probability $\mathbb{P}(Z \mid x)$
- joint mean $\mathbb{E}(Z \mid x)$

Each of these quantities is intractable to calculate exactly, in general.



Advanced generative models





Advanced generative models

- Simulation and variational methods are two broad classes of approaches to inverting models
- We'll explore these in the next couple classes

Example 1: Interacting particles

We have a graph with edges E and vertices V . Each node i has a random variable Z_i that can be “up” ($Z_i = 1$) or “down” ($Z_i = 0$)

$$\mathbb{P}_\beta(z_1, \dots, z_n) \propto \exp \left(\sum_{s \in V} \beta_s z_s + \sum_{(s,t) \in E} \beta_{st} z_s z_t \right).$$

This is called an “Ising model” and is central to statistical physics

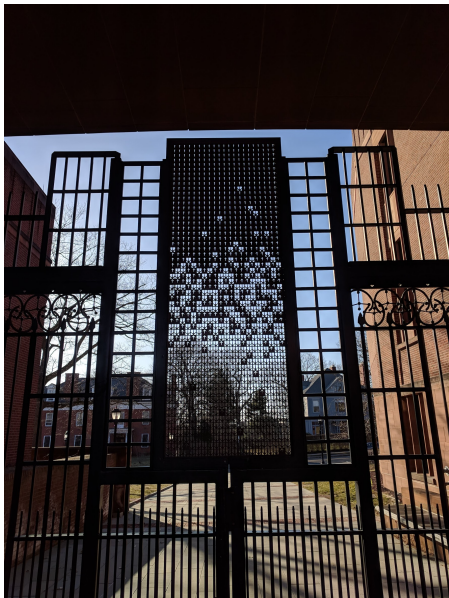
Example 1: Interacting particles

We have a graph with edges E and vertices V . Each node i has a random variable Z_i that can be “up” ($Z_i = 1$) or “down” ($Z_i = 0$)

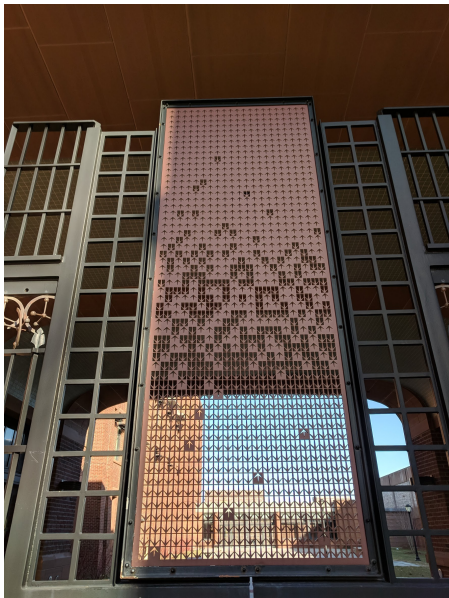
$$\mathbb{P}_\beta(z_1, \dots, z_n) \propto \exp \left(\sum_{s \in V} \beta_s z_s + \sum_{(s,t) \in E} \beta_{st} z_s z_t \right).$$

Imagine the Z_i are votes of politicians, and the edges encode the social network of party affiliations

Sterling Gate



Sterling Gate



Ising model

- Generative model for Z
- However, we can't sample from it directly
- Instead, we sample each component iteratively

Ising model, simple case

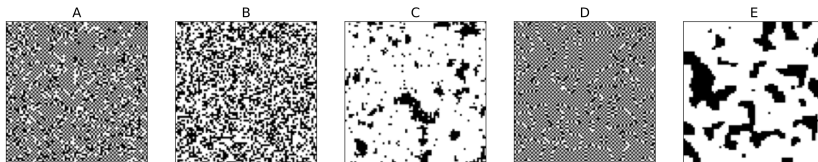
We have a graph with edges E and vertices V . Each node i has a random variable Z_i that can be “up” ($Z_i = 1$) or “down” ($Z_i = -1$)

$$\mathbb{P}_\beta(z_1, \dots, z_n) \propto \exp \left(\alpha \sum_{s \in V} z_s + \beta \sum_{(s,t) \in E} z_s z_t \right)$$

The sign of the parameter β effects whether neighboring states are encouraged to be the same ($\beta > 0$) or different ($\beta < 0$)

E are the set of edges, V are the vertices.

Ising model, simple case



- (c) The figure above shows the result of running Gibbs sampling on a 75×75 grid, using a fixed value of α and five different values of β :

$$\beta \in \left\{-5, -\frac{1}{2}, 0, \frac{1}{2}, 5\right\}$$

Which is which? Explain your answer.

Ising model is the “discrete Gaussian”

The nodes can take values in $\{0, 1\}$ or $\{-1, 1\}$ —The two encodings are equivalent (exercise)

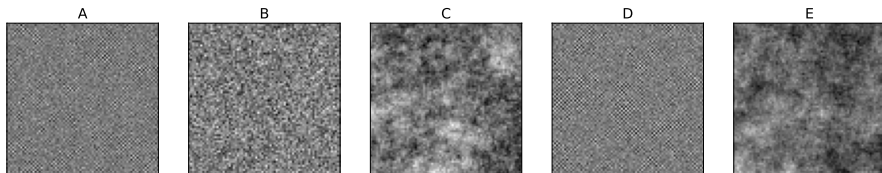
We can think of the Ising model as a “discrete Gaussian”.

Why? The multivariate Gaussian and Ising models have exactly the same form, but the sample spaces are different.

The normalizing constant for the multivariate Gaussian has a closed form; not so for the Ising model.

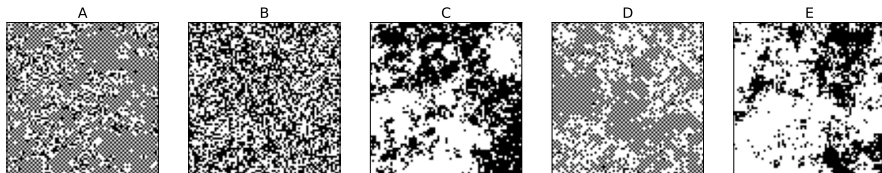
Ising model is the “discrete Gaussian”

Here is what we get when we sample Gaussians having the same parameters as in the midterm question:



Ising model is the “discrete Gaussian”

Here is what we get when we sample Gaussians having the same parameters as in the midterm question:



plotting the sign of z_s

Stochastic approximation

Gibbs sampler

Iterate until converged:

- 1 Choose vertex $s \in V$ at random
- 2 Sample z_s holding others fixed

$$\theta_s = \text{sigmoid} \left(\beta_s + \sum_{t \in N(s)} \beta_{st} z_t \right)$$
$$Z_s \mid \theta_s \sim \text{Bernoulli}(\theta_s)$$

You should verify that this is the correct conditional distribution of Z_s holding the others fixed.

Gibbs sampler

- 1 Choose vertex $s \in V$ at random
- 2 Sample $u \sim \text{Uniform}(0, 1)$ and update

$$z_s = \begin{cases} 1 & u \leq \left(1 + \exp\left(-\beta_s - \sum_{t \in N(s)} \beta_{st} z_t\right)\right)^{-1} \\ 0 & \text{otherwise} \end{cases}$$

- 3 Iterate

Resting state



Demo

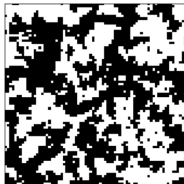
beta=1.50



beta=0.75



beta=0.50



beta=0.25



Demo

original

IML

noisy



Gibbs step: 100000



Demo

original



noisy



Gibbs step: 500000



Notes on Simulation

To learn more, see posted notes on simulation and Gibbs sampling

14.5 Why It Works

An understanding of why MCMC works requires elementary Markov chain theory, which is reviewed in an Appendix at the end of this chapter.

Recall that a distribution π satisfies *detailed balance* for a Markov chain if

$$p_{ij}\pi_i = p_{ji}\pi_j. \quad (14.57)$$

If π satisfies detailed balance, then it is a stationary distribution for the chain.

Because we are now dealing with continuous state Markov chains, we will change notation a little and write $p(x, y)$ for the probability of making a transition from x to y . Also, let's use $f(x)$ instead of π for a distribution. In this new notation, f is a stationary distribution if $f(x) = \int f(y)p(y, x) dy$ and detailed balance holds for f if

$$f(x)p(x, y) = f(y)p(y, x). \quad (14.58)$$

Variational methods

- Gibbs sampling is *stochastic* approximation
- Variational methods iteratively refine *deterministic* approximations
- We'll first discuss “mean field” approximations, originating in statistical physics

Deterministic approximation

Mean field variational algorithm

Iterate until converged:

- 1 Choose vertex $s \in V$ at random
- 2 Update mean μ_s holding others fixed

$$\mu_s = \text{sigmoid} \left(\beta_s + \sum_{t \in N(s)} \beta_{st} \mu_t \right)$$

Deterministic vs. stochastic approximation

- The z_s variables are random
- The μ_s variables are deterministic
- The Gibbs sampler convergence is in distribution
- The mean field convergence is numerical
- The Gibbs sampler approximates the full distribution
- The mean field algorithm approximates the mean of each node

Variational methods

To be continued next class!

Summary

- Approximation is required for many types of models
- Two forms: Simulation (Gibbs sampling) and variational methods
- Gibbs sampling iteratively makes stochastic approximations
- Variational methods iteratively make deterministic approximations